

PYTHON

- Un programma Python è una sequenza di definizioni e comandi
- L'interprete Python valuta le definizioni ed esegue i comandi
- La console un programma che permette di inserire i comandi di Python una riga alla volta
- Il testo che segue # non viene interpretato da Python, viene definito **commento**
- La **variabile** è il nome che assegniamo ad un oggetto caricato in memoria. Es: `x = 20` (la nostra variabile è x)
- Python permette le **assegnazioni** multiple su un'unica stringa
esempio: `x,y = 2, 3`
`print (x)`
[out]: `x = 2`
- Per **indentazione** intendiamo quella tecnica utilizzata attraverso la quale si evidenziano dei **blocchi** di programma con l'inserimento di una certa quantità di spazio vuoto all'inizio di una riga di testo.

❖ PRIMO PROGRAMMA:

```
x = 20      #x è una variabile e 20 è un'assegnazione
g = x
while abs(x - g * g) > 0.00001:
    print (g)
    g = 0.5*(g+x/g)
print (g)
```

[out]: 20

```
10.5
6.2023809523809526
4.713474545288365
4.478314445474382
4.4721402170657
4.47213595500161
```

➤ TIPI DI DATI

int – interi

Esempio: `x = 2`
 `y = 7`
 `z = x+y`
 `print (type (z))`
 [out]: **int**

string – stringa

Esempio: `x = 'testo'`
 `print (type (x))`
 [out]: **str**

Per scrivere del testo bisogna inserirlo tra ‘ ’

bool – booleani

Esempio: `x = 2`
 `y = 7`
 `z = x > y`
 `print (type (z))`
 [out]: **bool**

float – virgola mobile (decimali)

Esempio: `x = 2`
 `y = 7`
 `z = x+y`
 `print (type (z))`
 [out]: **float**

complex – numeri complessi

Esempio: `x = 3 + 0j`
 `print (type (3 + 0j))`
 [out]: **complex**

➤ FUNZIONI INCORPORATE

PRINT

`Print()` = stampa la stringa che inseriamo tra parentesi

Esempio: `x = 'programmazione'`
 `Print ('programmazione')`
 [out]: programmazione

ABS

La funzione `abs()` restituisce il valore assoluto del numero specificato.

Esempio: `x = 2`
 `y = - 4`
 `z = x + y`
 `print (abs(z))`
 [out]: 2

➤ CICLO WHILE

il **ciclo while** consiste nel ripetere una sequenza di istruzioni fino a quando una determinata condizione è verificata.

```
x = 1
while x < 5:
    print (x)
    x += 1
```

```
[out]: 1
      2
      3
      4
```

➤ INPUT

La funzione **input()** arresta l'esecuzione del programma e attende che l'utente digiti qualcosa, quello che viene digitato dall'utente viene preso, impacchettato in una stringa e restituito in output.

Esempio: `x = input('digita qualcosa: ')`
 `print (x)`
 [out]: digita qualcosa: [in]: programmazione
 [out]: programmazione

È importante ricordare che **input()** darà sempre una stringa. Anche se noi inseriamo un numero.

Quindi nel caso in cui l'utente inserisca un numero, dobbiamo trovare il modo di trasformare la stringa in un numero.

Questa cosa è possibile tramite le funzioni **float()** e **int()** che non fanno altro che tradurre il suo input in un float o in un int.

Da notare che ci sono comunque dei casi in cui queste funzioni “falliscono”. Se ad esempio l'utente invece di scrivere 12, scrive dodici, chiaramente queste funzioni non sono in grado di tradurre la stringa in numero.

Esiste chiaramente anche la funzione **str()** che converte un input in una stringa.

Esempio: `x = input ('Digita un numero')`
 `y = float (x)`
 `print (y)`
 [out]: digita un numero: [in]: 21
 [out]: 21.0
 [in]: type(y)
 [out]: float

➤ OPERATORI TRA TIPI NUMERICI

SOMMA

```
x = 2
y = 7
z = x + y
print (z)
```

[out]: 9

DIFFERENZA

```
x = 2
y = 7
z = x - y
print (z)
```

[out]: - 5

MOLTIPLICAZIONE

```
x = 2
y = 7
z = x * y
print (z)
```

[out]: 14

DIVISIONE REALE

```
x = 2
y = 7
z = x / y
print (z)
```

[out]: 0.2857142857142857

DIVISIONE INTERA

```
x = 2
y = 7
z = x // y
print (z)
```

[out]: 0

RESTO DELLA DIVISIONE

```
x = 2
y = 7
z = x % y
print (z)
```

[out]: 2

POTENZA DI UN NUMERO

```
x = 2
y = 7
z = x ** y
print (z)
```

[out]: 128

➤ FUNZIONI DI CONVERSIONE: FLOAT, INT (Casting)

Il casting è quell'operazione che restituisce il valore di un'espressione (o una variabile) convertito in un altro tipo. Difatti la variabile resta comunque del tipo originale.

```
Esempio:      x = float (20)
               print (x, type (x))

               y = int ('1234')
               print (y, type (y))
```

N.B: lo scorso anno ha parlato anche di conversioni str

```
Esempio:      z = str(3.14)
               print (type (z))
```

➤ NONE

None è un tipo di valore interno di Python che ci torna utile quando dobbiamo creare una variabile “senza valore”.

None può essere ovviamente usato anche all'esterno della funzione.

```
lista_valori = [1, 2, None, 4, None, 6]
```

```
for valore in lista_valori:
    if valore is not None:
        print(f'Valore: {valore}')
    else:
        print("Valore mancante.")
```

```
[out]: valore: 1
       valore: 2
       valore: mancante
       valore: 4
       valore: mancante
       valore: 6
```

➤ DIRAMAZIONE DEL CODICE: IF, ELIF, ELSE

Esercizio: Scriviamo un programma che chieda all'utente di inserire due stringhe che stampa la prima stringa, se la prima stringa è contenuta nella seconda stringa, stampa la seconda stringa, se la seconda stringa è contenuta nella prima. Se nessuna delle due cose è vera stampiamo un messaggio che indica che le due stringhe non si appartengono.

```
x= input('Digita la prima stringa ')
y= input('Digita la seconda stringa ')
    if x in y:
        print('La prima stringa è contenuta nella seconda')
        # se x è contenuto in y, allora proseguiamo con il blocco di istruzioni.
        #Se la condizione è falsa non vengono eseguite le istruzioni e si passa
        al codice successivo.

    elif y in x:
        print('La seconda stringa è contenuta nella prima')

    else:
        print('Le stringhe non si contengono')
```

Queste sono 3 condizioni:

IF = tradotto 'se'

ELIF = tradotto 'altrimenti se' → si può anche scrivere else if

ELSE = tradotto 'allora'

➤ ROUND

Esempio: round (3.1415)
 [out]: 3

 round (3.1415, 2)
 [out]: 3.14

⇒ Arrotonda il numero

➤ OPERATORI DI ASSEGNAZIONE

ASSEGNA

=

Esempio: $a = 5 \rightarrow$ assegna alla variabile a il valore 5

SOMMA E ASSEGNA

$+=$

Esempio: $a += 2$ equivale a fare $a = a + 2$

SOTTRAE E ASSEGNA

$- =$

Esempio: $a -= 2$ equivale a fare $a = a - 2$

MOLTIPLICA E ASSEGNA

$* =$

Esempio: $a *= 2$ equivale a fare $a = a * 2$

DIVIDE E ASSEGNA

$/ =$

Esempio: $a /= 2$ equivale a fare $a = a / 2$

DIVIDE E ASSEGNA

$// =$

Esempio: $a //= 2$ equivale a fare $a = a // 2$

CALCOLA IL RESTO E ASSEGNA

$\% =$

Esempio: $a \% 2$ equivale a fare $a = a \% 2$

CALCOLA LA POTENZA E ASSEGNA

$** =$

Esempio: $a ** 2$ equivale a fare $a = a^2$

UGUALE

$==$

Esempio: $x \% 2 == 0 \rightarrow$ verifica se i due membri sono uguali e stamperà vero o falso

➤ OPERATORI TRA STRINGHE

Concatenazione → somma tra solo stringhe (NO una stringa e un numero)

Esempio: x, y = 'python', 'programmazione'
 z = x+y
 print (z)
 [out]: pythonprogrammazione

Ripetizione → Se proviamo a moltiplicare una stringa per un numero, otteniamo una ripetizione della stringa. N.B: il numero che moltiplica deve essere un int e non un float

Esempio: x = 'python'
 x*2
 [out]: 'pythonpython'

Funzione len

Ci indica la lunghezza della stringa

Esempio: x = 'python'
 print ('la lunghezza di x è', len(x))
 [out]: la lunghezza di x è 6

Indicizzazione

Usiamo l'indicizzazione per estrarre da una stringa singoli caratteri.

Esempio: x = 'programmazione'
 print (x[0])
 print(x[3])
 print (x[-1]) #quando è negativa significa che partiamo dall'ultimo c
 arattere
 print (x[-3])

 [out]: p
 g
 e
 o

Appartenenza o in

l'operatore **in** usato tra due stringhe ci da come risultato True o False a seconda se la variabile a sinistra di in è contenuta nella variabile a destra di x

Esempio: x = 'python'
 a = 'tho'
 print(a in x)
 [out]: True

Esempio 2: y = 'programmazione'
 a = 'tho'
 print(a in y)
 [out]: False

➤ OPERATORI LOGICI - BOOLEANI

Operatore and → date due condizioni (A, B) e assegnato a loro il valore booleano and
La condizione sarà vera soltanto se entrambi i valori sono veri
In altri casi sarà falsa

Operatore or → date due condizioni (A, B) ed assegnato a loro il valore booleano or
La condizione sarà vera quando almeno una delle due condizioni è vera
La condizione sarà falsa soltanto quando entrambe le condizioni sono false

Operatore not → non fa altro che negare la condizione

```
c0, c1 = True, False
```

```
Print (c0 or c1)
```

```
[out]: True #stampa True perché c0 è true, basta che una delle due condizioni sia True
```

```
Print (not c1)
```

```
[out]: True #stampa True perché c1 è false, quindi diventa True.
```

```
Print (c0 and not c1)
```

```
[out]: True #perché c0 è true e not c1 è anche true.
```

➤ OPERATORI RELAZIONARI

Minore: <

Maggiore: >

Uguale: =

Minore uguale: <=

Maggiore uguale: >=

Diverso: !=

➤ IMMUTABILITÀ DELLE STRINGHE

```
Esempio: a = 'programmazione'
          a[2] = *
          [out]: Error
```

Non posso modificare i caratteri di una stringa, infatti **la stringa è detta immutabile**

Se abbiamo $y \rightarrow _ _ _ * _ *$ possiamo incrementare $y = y + '*'$
e avremo $_ _ _ * **$
⇒ Poiché la 1° stringa non viene modificata, ma viene copiata e poi incrementata

➤ LA DICHIARAZIONE FOR SULLE STRINGHE

For ci consente di iterare sopra una stringa carattere per carattere

Esempio: x = 'python'
 for a in x:
 print (a)

[out]: p
 y
 t
 h
 o
 n

➤ SLICING

Lo slicing ci consente di estrarre sottostringhe di lunghezza arbitraria.

- Esempio 1:

Name [0:2] == 'Bi'
Name [2:5] == 'ngo'
Name [:4] == 'Bing'
Name [:] == 'Bingo'
Name [1:-1] == 'ing'

0	1	2	3	4
B	I	N	G	O

S [START:END]

- Esempio 2:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
P	R	O	G	R	A	M	M	A	Z	I	O	N	E

x = 'programmazione'
Print(x[1:8:2]) = 'rgam'

S [START:STOP:STEP]

ESERCIZIO 1:

x = 'programmazione'
print(x[1:5])

[out]: rogr

➤ FUNZIONI

Le **funzioni** sono mini programmi che svolgono compiti specifici. Si riconoscono perché hanno un nome e delle parentesi dove inserire i dati → **nome_funzione (dato da inserire)**

Noi abbiamo in Python già delle funzioni preimpostate (interne del linguaggio), come la funzione **print ()** o la funzione **len ()**.

Se non sappiamo cosa fa una funzione, ci basta scrivere **help()** nella console.

Se vogliamo importare soltanto una funzione da una libreria ci basta scrivere → **from nome_libreria import nome_funzione**.

```
from math import sin
x = 1
seno = sin(x)
print (seno)

[out]: 0.8414709848078965
```

Per definire una funzione scriviamo **def nome_funzione (x,y)**

```
def media (a,b):
    c = (a+b)/2
    return c
n1=10
n2 = 20
print (media(n1,n2))

[out]: 15.0
```

L'istruzione **return** è una funzione che può restituire un valore e terminare l'esecuzione della funzione

```
x = input ('digita un numero')

def verifica_pari (x):
    if x % 2 == 0:
        return True
    else:
        return False

risultato = verifica_pari(int(x))
print(risultato)
```

[out]: in base al numero che inseriamo il programma ci stamperà True o False

COMPLESSITA' COMPUTAZIONALE → l'operatore che fornisce il numero di operazioni necessarie a risolvere un determinato problema in funzione del numero di dati da trattare, usando l'algoritmo più efficiente possibile.

COSTO COMPUTAZIONALE → è il costo definito in termini di risorse di calcolo
COMPLESSITA' TEMPORALE → è il tempo impiegato dal codice per eseguire le operazioni. Questo dipende dalla lunghezza delle stringhe.

COMPLESSITA' SPAZIALE → è l'utilizzo delle risorse da parte di un programma

➤ NOTAZIONE ASINTOTICA

O (GRANDE) → Caso peggiore

Lineare	$O(n)$
Costante	$O(1)$

θ (TETA) → Caso Medio

```
a = input('inserisci un testo ')

# voglio sapere la complessità temporale da qui in poi

n = len(a)

if 'A' in a:
    print('Si')
else:
    print('No')

# complessità è Theta(n) nel caso peggiore, 'A' non in a
# è Theta(n) anche nel caso medio.
```

➤ LE TUPLE

Le tuple vengono utilizzate per memorizzare più elementi in una singola variabile. Posso definirla come una raccolta ordinata e immutabile (cioè non posso modificare o aggiungere o rimuovere elementi una volta creata) dove posso trovare valori duplicati. Le tuple si scrivono tra parentesi tonde e può contenere elementi di tipo diverso.

```
t = (1, 2.3, 'python', True)
```

Tutte le operazioni che abbiamo visto per le stringhe posso applicarle anche alle tuple. Quindi posso fare l'indicizzazione, lo slicing, la concatenazione e posso applicare l'operazione di ripetizione.

```
t = (1, 2.3, 'python', True, 1)
```

#tipo della tupla

```
print (type (t))  
[out]: tuple
```

#lunghezza della tupla

```
print (len (t))  
[out]: 5
```

#indicizzazione

```
print (t[2])  
[out]: python
```

#slicing

```
t0 = t [1::2]  
print (t0)  
[out]: (2.3, True)
```

#concatenazione

```
t0 = (1, 2, 3)  
t1 = ('uno', 2, 'III')  
t = t0 + t1  
print (t)  
[out]: (1, 2, 3, 'uno', 2, 'III')
```

#ripetizione

```
t = 2*t0  
print (t)  
[out]: (1, 2, 3, 1, 2, 3)
```

Se ho una tupla contenente due parole posso estrarre il carattere di una parola facendo la seguente operazione

```
t = ('python', 'programma')  
print (t[0][1])
```

```
[out]: y
```

Un'operazione che posso fare sulle tuple e non sulle stringhe è l'operazione di **unpacking**.

Un'altra operazione è l'operazione di **packing**.

Possiamo convertire una tupla in una lista con la seguente operazione

```
t = (1, 2.3, 'python', True, 1)  
l = list ( t )  
Print ( l )
```

```
[out]: [1, 2.3, 'python', True, 1]
```

➤ LE LISTE

Le liste sono sequenze di elementi (come una tupla). Le liste sono mutabili. Quindi una **lista** aggrega dei valori di qualsiasi tipo, racchiusi tra una coppia di parentesi quadre e separati da una virgola.

<code>l₀ = []</code>	<code>#lista vuota</code>
<code>l₁ = [9, 'python', 3.14]</code>	<code>#lista con una serie di elementi di tipo diverso</code>
<code>l₂ = [[1,2], [2,3], [6,7]]</code>	<code>#lista che contiene altre liste</code>

La lista supporta le stesse operazioni delle stringhe, ovvero: indicizzazione, slicing, concatenazione, ripetizione, funzione len...e così via.

```
l = [1, 2.3, 'python', True, [1,9], (1,2)]
```

#tipo della lista

```
print (type (l))  
[out]: list
```

#lunghezza della tupla

```
print (len (l))  
[out]: 6
```

#indicizzazione

```
print (l[2])  
[out]: python
```

#slicing

```
l0 = l [1::2]  
print (l0)  
[out]: (2.3, True, (1,2))
```

#concatenazione

```
l0 = (1, 2, 3)  
l1 = ('uno', 2, 'III')  
l = l0 + l1  
print (l)  
[out]: (1, 2, 3, 'uno', 2, 'III')
```

#ripetizione

```
l = 2*l0  
print (l)  
[out]: (1, 2, 3, 1, 2, 3)
```

#cambiare elemento all'interno di una lista

```
l[2] = 'una stringa'  
print (l)  
[out]: [1, 2.3, 'una stringa', True, [1, 9], (1,2))
```

Un'operazione che posso fare sulle liste e non sulle altre strutture è l'operazione `del`, che ci permette di eliminare un elemento dalla lista.

```
l = [1, 2.3, 'python', True, [1,9], (1,2)]
del ( l [1])
Print (l)

[out]: l = [1, 'python', True, [1,9], (1,2)]
```

Un'altra operazione è l'operazione di **append**, che ci permette di aggiungere elementi alla lista

```
l = [1, 2.3, 'python', True, [1,9], (1,2)]
l.append ('xxx')          #nome_lista.append
print ( l )

[out]: l = [1, 2.3, 'python', True, [1,9], (1,2), 'xxx']
```

Possiamo convertire una lista in una tupla con la seguente operazione

```
l = [1, 2.3, 'python', True, 1]
t = tuple ( l )
print ( t )

[out]: (1, 2.3, 'python', True, 1)
```

Il metodo **index ()** viene utilizzato nelle liste per trovare l'indice di un elemento specificato all'interno di una sequenza.

(N:B: restituisce la prima occorrenza; se ci stanno 2 elementi uguali)

```
l = [1, 2.3, 'python', True, 1]
z = l.index ( 1 )
print ( z )

[out]: 0
```


Il metodo **join ()** è un metodo delle stringhe e viene utilizzato per concatenare gli elementi di una sequenza in una stringa.

```
l = [ 'Questo' , 'è' , 'un' , 'esempio' ]  
s = ' '.join( l )  
print ( s )
```

[out]: Questo è un esempio

Il metodo **extend ()** è un metodo utilizzato per estendere una lista aggiungendo tutti gli elementi di un'altra lista o di un'altra sequenza iterabile alla fine della lista corrente

```
l1 = [ 1, 2, 3 ]  
l2 = [ 4, 5, 6 ]  
l1.extend l2  
print ( l1 )
```

[out]:[1, 2, 3, 4, 5, 6]

➤ FUNZIONI INCORPORATE: RANGE, MIN E MAX

La funzione **range ()** in Python restituisce una sequenza di numeri interi partendo dal valore iniziale specificato fino al valore finale specificato. La sua sintassi è

range (start, stop, step)

dove start è il valore iniziale, stop è il valore finale non incluso nella sequenza e step è il passo incrementato.

La funzione **min()** in Python restituisce il valore minimo in una sequenza di numeri o caratteri.

La funzione **max()** in Python restituisce il valore massimo in una sequenza di numeri o caratteri.

```
L = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for x in range (min(L), max(L)-1):
    print(x)
```

```
[out]: 1
        2
        3
        4
        5
        6
        7
        8
```

➤ BREAK

L'istruzione **break** permette di uscire dal ciclo, interrompe l'intero ciclo nella posizione in cui si trova. Altera il flusso del codice non è dipendente da una condizione.

```
numero = 0
while numero < 5:
    print (numero)
    if numero == 3:
        break
    numero += 1
```

```
[out]: 0
        1
        2
        3
```

➤ FUNZIONE ZIP

La funzione `zip ( )` in python combina insieme elementi prese da sequenze diverse.

```
a = [ 3, 4, 5, 6 ]  
b = [ 'tre', 'quattro', 'cinque', 'sei' ]
```

```
for x in zip ( a, b ):  
    print( x )
```

```
[out]: ( 3, 'tre' )  
       ( 4, 'quattro' )  
       ( 5, 'cinque' )  
       ( 6, 'sei' )
```

#altro modo

```
a = [ 3, 4, 5, 6 ]  
b = [ 'tre', 'quattro', 'cinque', 'sei' ]
```

```
x = list (zip ( a, b )  
          print( x )
```

```
[out]: [( 3, 'tre' ), ( 4, 'quattro' ), ( 5, 'cinque' ), ( 6, 'sei' )]
```

L'operatore di spaccettamento `*` viene utilizzato per estrarre gli elementi da una sequenza e passarli come argomenti separati a una funzione o a un'istruzione di stampa. In questo caso, `*x` viene utilizzato per estrarre gli elementi dalla tupla `x` generata dalla funzione `zip()` e stamparli come argomenti separati

```
a = [ 3, 4, 5, 6 ]  
b = [ 'tre', 'quattro', 'cinque', 'sei' ]
```

```
for x in zip ( a, b ):  
    print( *x )
```

```
[out]: 3 tre  
       4 quattro  
       5 cinque  
       6 sei
```

➤ BUBBLE SORT

È un algoritmo di ordinamento che confronta e scambia ripetutamente coppie di elementi se sono fuori ordine. Nell'esempio con il bubble sort andremo ad ordinare la lista

```
x = [20, 10, 9, 8, 7, 6, 5, 4, 3, 100]
```

```
def bubble_sort( a ):
```

```
    n = len(a)
```

```
    for c in range(n-1):
```

```
        for i in range(n-1-c):
```

```
            if a[i] > a[i+1]:
```

```
                a[i], a[i+1] = a[i+1], a[i]
```

```
bubble_sort(x)
```

```
print (x)
```

```
[out]: [3, 4, 5, 6, 7, 8, 9, 10, 20, 100]
```

#ha complessità teta di (n*n)

➤ RICORSIONE E MERGE SORT

La ricorsione sono funzioni che chiamano loro stesse.

```
lista=[1,2,10,[5,7,81],1,42,3]

def massimo_ricorsivo(lista):
    massimo=0
    for elemento in lista:
        if type(elemento)==int and elemento > massimo: massimo=elemento
        elif type(elemento)==list:
            if massimo_ricorsivo(elemento) > massimo: massimo =
                massimo_ricorsivo(elemento)
    return massimo

print(massimo_ricorsivo(lista))
```

La funzione `massimo_ricorsivo` prende in input una lista e restituisce il valore massimo all'interno della lista. La funzione esamina ogni elemento della lista e, se l'elemento è un intero, lo confronta con il valore massimo attualmente noto. Se l'elemento è una lista, la funzione chiama se stessa (in modo ricorsivo) per trovare il valore massimo all'interno della lista.

Il **Merge Sort** è un algoritmo ricorsivo di sorting (ordina una lista di elementi). Molto veloce. Molto usato.

L'idea di fondo del merge sort è che:

- non è veloce ordinare una lista disordinata
- è molto veloce, invece, mettere insieme due liste già ordinate

```
def merge_sort(lista):
    if len(lista)>1:      #fino a che non posso più spezzare la lista
        merge_sort(meta_lista_di_sinistra)
        merge_sort(meta_lista_di_destra)
    merge(meta_lista_di_sinistra, meta_lista_di_destra)
```

L'idea di base è quella di dividere la lista a metà, ordinare le due metà separatamente e poi unirle in modo ordinato. Questo processo viene ripetuto fino a quando ogni metà contiene solo un elemento.

```
lista_da_ordinare=[1,6,2,9,4,3,8,7]
```

```
merge_sort([1,6,2,9,4,3,8,7]):  
    merge_sort([1,6,2,9])  
    merge_sort([4,3,8,7])
```

```
merge_sort([1,6,2,9]):  
    merge_sort([1,6])  
    merge_sort([2,9])
```

```
merge_sort([1,6]):  
    merge_sort(1)    #le ho srotolate fino a che le liste non sono di un elemento  
    merge_sort(6)  
    merge([1],[6])
```

```
merge_sort([2,9]):  
    merge_sort(2)  
    merge_sort(9)  
    merge([2],[9])
```

Il tempo richiesto per ordinare una lista di lunghezza n è:

$$T(\text{totale}) = T(\text{merge}) * \text{numeroDiLivelli}$$

Dove:

$$T(\text{merge}) = O(n)$$

I livelli sono tanti quante le volte che posso dividere n per 2 cioè... $\log_2(n)$

Quindi:

$$T(\text{totale}) = T(\text{merge}) * \text{numeroLivelli} = O(n) * \log_2(n) = O(n * \log(n))$$

➤ RICERCA BINARIA

La ricerca binaria consiste nel cercare la posizione di un elemento in una sequenza cominciando dall'elemento in mezzo.

```
a = [2, 4, 6, 7, 7, 10, 10, 10, 10, 14, 16, 17, 20, 23, 23]
```

```
def ricerca_binaria(a, k):  
    sx, rx = 0, len(a)  
  
    while rx > sx:  
        cx = (sx + rx) // 2  
        if a[cx] == k:  
            return cx  
        if k < a[cx]:  
            rx = cx  
        else:  
            sx = cx + 1  
  
    return -1  
  
print (ricerca_binaria(a, 10))
```

```
[out]: 7
```

➤ DEEP_COPY

La funzione `deep_copy`

```
a = [3, [4, 5, 6, [10, 11]], 7, 8, [12, 13, [2, 3]]]

def deep_copy(a):
    b = []
    for e in a:
        if type(e) == list:
            b.append(deep_copy(e))
        else:
            b.append(e)

    return b

b = deep_copy(a)

print("Lista originale:", a)
print("Copia profonda:", b).
```

Una funzione `lambda` è una piccola funzione anonima. La sintassi è
Lambda argomento: espressione

```
quadrato = lambda x: x**2
print(quadrato(5))
```

[Out]: 25

La funzione **`sorted ( )`** e il metodo **`sort ( )`** riordinano entrambi una lista

```
n = [ 5, 2, 8, 1, 3 ]
b = sorted ( n )
print ( b )
```

[Out]: [1, 2, 3, 5, 8]

```
n = [ 5, 2, 8, 1, 3 ]
b = sorted ( n )
print ( b )
```

[Out]: [1, 2, 3, 5, 8]

➤ I DIZIONARI

Un dizionario è un insieme di elementi che sono composti da due entità: una chiave e un valore.

Ogni chiave è associata ad un valore. Le chiavi nel dizionario sono uniche. I dizionari sono mutabili cioè posso modificarli dopo la creazione. Vengono creati con delle parentesi graffe e la coppia chiave – valore è separata da :

```
Esempio 1: dizionario = {  
                'nome': 'Alice',  
                'python': 5,  
                3 : 'ingegnere',  
                3: 3.14  
            }
```

```
Esempio 2: dizionario = {'nome': 'Alice', 'python': 5, 3 : 'ingegnere', 3: 3.14 }
```

Un dizionario supporta le seguenti operazioni:

```
dizionario = { "nome" : "Alice", "python" : 5, 3 : "ingegnere", 3:3.14 }
```

#lunghezza

```
print (len(dizionario))  
[out]: 3
```

#leggere il valore di una chiave

```
print (dizionario['python'])  
[out]: 5
```

#modificare il valore associato ad una chiave

```
dizionario[3] = 'informatico'  
print (dizionario [3])  
[out]: informatico
```

#inserire una nuova coppia chiave-valore

```
dizionario ['NOME'] = dizionario['nome']  
print(dizionario)  
[out]: {'nome': 'Alice', 'python': 5, 3: 'informatico', 'NOME': 'Alice'}
```

#cancellare un elemento del dizionario

```
del(dizionario[3])  
print (dizionario)  
[out]: {'nome': 'Alice', 'python': 5, 'NOME': 'Alice'}
```

Possiamo iterare all'interno di un dizionario, attraverso le chiavi o i valori, tramite un ciclo for:

```
dizionario = { "nome" : "Alice", "python" : 5, 3 : "ingegnere", 4:3.14 }

for k in dizionario:
    print(k, dizionario[k])

[out]: nome Alice
       Python 5
       3 ingegnere
       4 3.14
```

Operatore **in**

```
dizionario = { "nome" : "Alice", "python" : 5, 3 : "ingegnere", 4:3.14 }

if 'Nome' in dizionario:
    x = dizionario['Nome']
else:
    x = 0
```

La funzione **chr ()** in python restituisce il carattere corrispondente al codice Unicode specificato.

```
codice_unicode = 65
carattere = chr(codice_unicode)
print(carattere)
```

[out]: A

La funzione **ord ()** in python restituisce il valore intero corrispondente al carattere specificato.

```
carattere = 'A'
valore_intero = ord(carattere)
print(valore_intero)
```

[out]: 65

Funzione **Open ()** ci permette di aprire un file in un programma. Questa funzione ci restituisce una struttura dati e ci permette di leggere il contenuto di un file di testo riga per riga.

```
def minuscolo( x ):

    return chr(ord(x) + ord('a') - ord('A'))

f = open('promessi_sposi.txt')
d = {}

for line in f:
    for x in line:
        if x in ['è', 'é']:
            x = 'e'
        elif x in ['ò']:
            x = 'O'
        if (x >= 'a' and x <= 'z') or (x >= 'A' and x <= 'Z'):
            if (x >= 'A' and x <= 'Z'):
                x = minuscolo(x)
            if x in d:
                d[x] += 1
            else:
                d[x] = 1

lista_d = []
for k in d:
    lista_d.append((k, d[k]))

lista_d.sort(key = lambda t: t[1], reverse=True)
```

Il metodo **get ()** restituisce il valore associato ad una determinata chiave del dizionario

```
studenti = { 'Alice': 25, 'Bob': 30, 'Charlie': 22, 'David': 28 }
età_alice = studenti.get ( 'Alice' )

print( "l'età di Alice:", età_alice)

[out]: [ 'l'età di Alice: 25' ]
```

Usiamo il metodo **items()** per restituire una lista di tuple che rappresentano le coppie chiave-valore del dizionario d.

```
d = {'a': 1, 'b': 2, 'c': 3}

lista_d = []
for t in d.items():
    lista_d.append(t)

print(lista_d)

[out]: [ ( 'a', 1 ), ( 'b', 2 ), ( 'c', 3 ) ]
```

Il metodo **copy()** quando vogliamo avere una copia di un determinato dizionario.

```
persona = {"nome": "Alice", "età": 30}
persona = persona.copy()

print (persona)
print (persona.copy())

[out]: {'nome': 'Alice', 'età': 30}
       {'nome': 'Alice', 'età': 30}
```

Il metodo **values()** restituisce una lista di tutti i valori del dizionario

```
studenti = { 'Alice': 25, 'Bob': 30, 'Charlie': 22, 'David': 28 }
età_studenti = list (studenti.values())

print("Età degli studenti:", età_studenti)

[out]: [25, 30, 22, 28]
```

Il metodo **keys()** restituisce una lista di tutte le chiavi del dizionario

```
studenti = { 'Alice': 25, 'Bob': 30, 'Charlie': 22, 'David': 28 }
nomi_studenti = list (studenti.keys())

print( "Nomi degli studenti:", nomi_studenti)

[out]: [ 'Alice', 'Bob', 'Charlie', 'David' ]
```

Complessità operazioni su dizionari

`d = {}` # $O(1)$

`n = len(d)`

`d['k'] = 10` # $O(1)$ caso medio sia inserimento che sia
aggiornamento caso peggiore $O(n)$

`'k' in d` # $O(1)$ nel caso medio, caso peggiore $O(n)$

`del(d['k'])` # $O(1)$ nel caso medio, caso peggiore $O(n)$

`d.get(k)` # $O(1)$ nel caso medio, caso peggiore $O(n)$

`for x in d:` # $\Theta(n)$
 `print(x)`

`for x in d.items():` # $\Theta(n)$
 `print(x)`

`for x in d.keys():` # $\Theta(n)$
 `print(x)`

`for x in d.values():` # $\Theta(n)$
 `print(x)`